Java – Lecția 9: Test (răspunsuri)

Întrebări 1–24 și două sarcini practice

1. Tipuri de date

Primitive: byte (`byte b = 10;`), short (`short s = 1000;`), int (`int x = 5;`), long (`long l = 10000000000L;`), float (`float f = 3.14f;`), double (`double d = 3.14;`), char (`char c = 'A';`), boolean (`boolean ok = true;`).

De referință: String (`String s = "Salut";`), tablouri, clase, interfețe, enum.

2. Variabila

Un spațiu de memorie cu nume, în care se păstrează o valoare. Declarare: `tip nume = valoare;`, de exemplu `int age = 15;`.

3. Cicluri

`for`, `while`, `do...while` și `for-each` (`for (int x : arr)`).

4. Operatori condiționali

`if`, `if-else`, `else if`, `switch-case` și operatorul ternar `? :`. Pentru comparare: `== != < > <= >=`; pentru combinare: `&& || !`.

5. Matrice (array)

O structură de lungime fixă, cu elemente de același tip, accesate prin index începând de la 0. Creare: `int[] a = new int[5];` sau `int[] a = {1, 2, 3};`.

6. Lungimea matricei

Câmpul `a.length` (nu este metodă, fără paranteze). Pentru String se folosește metoda `s.length()`.

7. Matrice multidimensională

O matrice de matrice: `int[][] m = new int[3][4];` sau `int[][] m = {{1, 2}, {3, 4}};`.

8. Clasa

Un șablon (plan) pentru crearea obiectelor; descrie câmpurile (starea) și metodele (comportamentul). Obiectul este o instanță a clasei: `Car c = new Car();`.

9. Metoda

Un bloc de cod cu nume, care execută o acțiune, poate primi parametri și returna o valoare. Exemplu: `int sum(int a, int b) { return a + b; }`.

10. Recursia

Situația în care o metodă se apelează pe ea însăși. Necesită un caz de bază (condiție de oprire), altfel apare `StackOverflowError`. Exemplu: `int f(int n) { return n <= 1 ? 1 : n * f(n - 1); }`.

11. Principiile OOP

Încapsularea, moștenirea, polimorfismul și abstractizarea.

12. Explicarea principiilor OOP

Încapsularea – datele sunt ascunse (`private`) și accesate prin getteri și setteri.

Moștenirea – o clasă preia câmpurile și metodele alteia (`extends`).

Polimorfismul – aceeași metodă se comportă diferit (suprascriere `@Override` și supraîncărcare).

Abstractizarea – se arată doar esențialul, detaliile se ascund (clase abstracte, interfețe).

13. Modificatori de acces

`public`, `protected`, default (package-private, fără cuvânt cheie) și `private`.

14. Metode din clasa String

`length()`, `charAt(int)`, `substring(int, int)`. Alte exemple: `equals()`, `toUpperCase()`, `indexOf()`, `split()`, `contains()`.

15. Gestionarea excepțiilor

Se folosește `try { ... } catch (Exception e) { ... } finally { ... }`. `throw` aruncă o excepție, iar `throws` o declară în semnătura metodei. Excepțiile sunt verificate (checked, ex. `IOException`) sau neverificate (unchecked, ex. `NullPointerException`).

16. Clasa abstractă

O clasă declarată cu `abstract`, din care nu se pot crea obiecte direct. Poate conține metode abstracte (fără corp), metode obișnuite și câmpuri. Se moștenește cu `extends`.

17. Interfața

Un contract care descrie ce metode trebuie să aibă o clasă (`interface`). Clasa o implementează cu `implements`. Metodele sunt implicit `public abstract`; de la Java 8 pot exista și `default` și `static`. Variabilele sunt constante.

18. Clasă abstractă vs. interfață

Clasa abstractă: se moștenește o singură dată, poate avea constructor, câmpuri obișnuite și orice modificatori.

Interfața: se pot implementa mai multe, nu are constructor, are doar constante și descrie capabilități.

19. Java Enum

Un tip special pentru un set fix de constante, ex. `enum Day { MON, TUE, WED }`. Poate avea câmpuri, constructori și metode. Metode utile: `values()`, `valueOf()`, `ordinal()`.

20. Biblioteci

`java.util` (ArrayList, HashMap, Scanner, Random), `java.io`, `java.time`, `java.lang` (Math, String), `javax.swing`, JUnit. Altele: `java.awt`, `java.net`, `java.sql`.

21. Biblioteca

Un set de clase și metode gata făcute, grupate în pachete, pe care le putem refolosi în loc să scriem codul de la zero. Se conectează cu `import`.

22. Metode folosite în testare (JUnit)

Adnotări: `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`.

Aserțiuni: `assertEquals()`, `assertTrue()`, `assertFalse()`, `assertNotNull()`, `assertThrows()`.

23. JButton

O clasă Swing care reprezintă un buton într-o interfață grafică. Exemplu: `JButton b = new JButton("OK");`. Acțiunea la apăsare se tratează cu `addActionListener(...)`.

24. Pentru ce este folosit Swing

O bibliotecă (`javax.swing`) pentru crearea interfețelor grafice desktop. Conține componente precum `JFrame`, `JPanel`, `JLabel`, `JButton`, `JTextField`.

Task 1 – citirea a 10 numere și sortarea crescătoare

```
import java.util.Scanner;

public class SortArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] a = new int[10];
        System.out.println("Introduceți 10 numere:");
        for (int i = 0; i < a.length; i++) {
            a[i] = sc.nextInt();
        }
        // sortare prin selecție
        for (int i = 0; i < a.length - 1; i++) {
            int min = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[min]) min = j;
            }
            int tmp = a[i]; a[i] = a[min]; a[min] = tmp;
        }
        for (int x : a) System.out.print(x + " ");
        // varianta scurtă: java.util.Arrays.sort(a);
    }
}
```

Task 2 – ștergerea tuturor aparițiilor unui număr

Matricea are lungime fixă, deci se creează una nouă, fără acel număr. Prima parcurgere numără elementele rămase, a doua le copiază – astfel nu rămân apariții ale numărului.

```
import java.util.Arrays;

public class RemoveValue {
    static int[] remove(int[] arr, int val) {
        int count = 0;
        for (int x : arr) if (x != val) count++;
        int[] res = new int[count];
        int k = 0;
        for (int x : arr) if (x != val) res[k++] = x;
        return res;
    }

    public static void main(String[] args) {
        int[] a = {3, 1, 3, 5, 3, 7};
        System.out.println(Arrays.toString(remove(a, 3))); // [1, 5, 7]
    }
}
```